TryHackMe — How The Web Works Path

How Websites Work

Learning Notes — HTML, JavaScript, Sensitive Data & HTML Injection

> ***"To exploit a website, you first need to know how it was built — and every website in the world is built with the same three technologies: HTML, CSS, and JavaScript."***

How Sites Work | HTML | JavaScript | Sensitive Data | HTML Injection

1. How Websites Work — The Big Picture

When you visit a website, your browser acts as the client, sending a request to a web server — a dedicated computer somewhere in the world running server software. The server receives the request, processes it, and responds with the data your browser needs to display the page. This request-response cycle underpins every website interaction on the internet.

Component	Front End (Client-Side)	Back End (Server-Side)
What it is	The part of the website your browser downloads and renders — what you see.	The server infrastructure that processes requests and generates responses.
Where it runs	In your browser on your device.	On the web server — you never directly see this code.
Technologies	HTML (structure), CSS (style), JavaScript (interactivity).	Python, PHP, Node.js, Ruby, databases (MySQL, MongoDB), etc.
Who sees it	Anyone — source code is publicly viewable via 'View Page Source'.	Nobody — server-side code is never sent to the browser.
Security risk	Source code is exposed — sensitive data left here is publicly readable.	Vulnerabilities here affect server data, authentication, databases.

Front End vs Back End — The Security Boundary

Everything in the front end (HTML, CSS, JavaScript) is sent to the user's browser and is publicly readable — there is no such thing as a secret in client-side code. Back-end code runs on the server and is never sent to the browser. This distinction defines the security boundary: never put credentials, API keys, tokens, or sensitive logic in front-end code.

1.1 The Three Building Blocks of Every Website

Technology	Role	Analogy
HTML	Structure & Content	The skeleton — defines what elements exist on the page and their content (headings, paragraphs, images, links, buttons).
CSS	Styling & Presentation	The skin — controls colours, fonts, layout, spacing, animations. Makes the skeleton look good.
JavaScript	Interactivity & Behaviour	The muscles — adds dynamic behaviour, responds to user actions, updates the page without reloading.

2. HTML — HyperText Markup Language

HTML (HyperText Markup Language) is the backbone of every web page. It defines the structure and content of a page using elements (also called tags). Browsers interpret HTML and render it as the visual page you see. Understanding HTML is the first step to both building and attacking web applications.

2.1 Standard HTML Document Structure

```
<!DOCTYPE html>
<!-- Tells the browser this is HTML5. Ensures consistent rendering across
browsers. -->

<html>
<!-- Root element — all other elements must be inside this. -->

  <head>
  <!-- Contains metadata — page title, CSS links, charset. NOT displayed
to the user. -->
    <title>TryHackMe</title>
  </head>
```

```
  <body>
  <!-- Everything the user sees goes here. -->

    <h1>Welcome to TryHackMe!</h1>
    <!-- Heading level 1 — the most important heading on the page -->

    <p>Welcome to my website.</p>
    <!-- Paragraph text -->

    <img src='img/cat-1.jpg'>
    <!-- Image — src points to the file path or URL of the image -->

    <a href='https://tryhackme.com'>Click me!</a>
    <!-- Anchor / hyperlink — href specifies the destination URL -->

    <button onclick='document.getElementById("demo").innerHTML =
"Clicked!";'>
      Click Me!
    </button>
    <!-- Button with onclick event — triggers JavaScript when clicked -->

  </body>
</html>
```

2.2 Common HTML Elements

Tag	Element Type	Purpose & Key Attributes
`<!DOCTYPE html>`	Document declaration	Must be the first line. Tells the browser the document uses HTML5 standard.
`<html>`	Root element	Contains the entire HTML document. All other elements are children of this.
`<head>`	Metadata container	Contains page title, charset, CSS/JS links, meta tags. Not visible to users.
`<title>`	Page title	Sets the text shown in the browser tab and bookmarks.
`<body>`	Content container	All visible content must be inside the body. Only content here is rendered.
`<h1>-<h6>`	Headings	Six levels of headings. h1 = most important (largest), h6 = least. Used for document structure.
`<p>`	Paragraph	Block of text. Browsers add spacing above and below by default.

<img>	Image	Displays an image. Key attributes: src (file path/URL), alt (accessibility text).
<a>	Anchor / Hyperlink	Creates a clickable link. Key attribute: href (destination URL). Can link to pages, files, or anchors.
<button>	Button	Clickable button. onclick attribute triggers JavaScript on click.
	Division / Container	Generic block container — used to group and style sections of content.
	Inline container	Generic inline container — used to style or identify a small section of text.
<input>	Input field	Accepts user input. type attribute controls the kind: text, password, email, checkbox, etc.
<form>	Form	Container for input elements. action specifies where form data is sent; method specifies GET or POST.
<!-- -->	Comment	Not rendered by browser. Visible in page source. NEVER put sensitive data in comments!

2.3 HTML Attributes — class, id, and More

HTML elements can have attributes that provide additional information or styling hooks. Two of the most important are class and id:

Attribute	Syntax Example	Rules & Use Cases
class	<p class="bold-text">	Multiple elements can share the same class. Used to apply CSS styles to a group of elements. Non-unique.
id	<p id="example">	Unique per page — only one element can have a given id. Used by CSS for specific styling and by JavaScript to target specific elements (document.getElementById).
src	<img src="img/cat.jpg">	Source path for images, scripts, and iframes. Must include correct file extension (e.g. .jpg, .png). Wrong path = broken image.
href	<a href="https://thm.com">	Hyperlink reference — the URL the anchor tag points to. Can be absolute (full URL) or relative (/about).
onclick	<button onclick="myFunc()">	Triggers JavaScript function when element is clicked. Inline event handler — can be used to call any JS function.

2.4 Lab — HTML Cat Website

The room includes a live HTML editor where you work with a cat-themed website. Two tasks test your HTML knowledge:

Task	What You Did	What It Taught
1	Fixed a broken image by adding the missing .jpg extension to the src attribute.	img src must include the correct file path and extension. A missing or wrong extension silently breaks the image. Answer: HTMLHERO
2	`<img src="img/dog-1.png">`	Adding a new img tag with correct path renders a new image on the page. No server restart needed — changes render immediately in the editor.

View Page Source — The Attacker's First Move

Right-click any web page → 'View Page Source' (Chrome) or 'Show Page Source' (Safari) to see the raw HTML the server sent. This is publicly accessible to anyone — no tools required. Every security assessor checks page source on first contact with a web application. Any credentials, tokens, comments, or hidden links in the HTML are immediately visible.

3. JavaScript (JS)

JavaScript (JS) is the programming language of the web. Unlike HTML (a markup language that describes structure) and CSS (a style language), JavaScript is a full programming language that enables websites to be interactive and dynamic. It is one of the most widely used programming languages in the world.

JavaScript — The Interactivity Engine of the Web

- JavaScript runs in the browser (client-side) — no server needed. Every major browser has a built-in JS engine.
- JS can dynamically update page content without reloading — this is how single-page apps (SPAs) work.
- JS responds to user events: clicks, keypresses, form submissions, mouse movements, page loads.
- JS manipulates the DOM (Document Object Model) — the browser's live tree representation of the HTML.
- JS can make HTTP requests to servers (AJAX/Fetch API) and update the page with the response.
- JS can be embedded directly in HTML (<script> tags) or loaded from external .js files.
- JavaScript is also widely used server-side via Node.js — powers the back end of many web apps.

3.1 Embedding JavaScript in HTML

JavaScript can be included in a web page in two ways:

```
<!-- Method 1: Inline JS in <script> tags (inside the HTML file) -->
<script>
  // This changes the content of any element with id='demo'
  document.getElementById('demo').innerHTML = 'Hack the Planet';
</script>

<!-- Method 2: External JS file (loaded via src attribute) -->
<script src='/path/to/script.js'></script>
<!-- External files are cached by browsers and keep HTML cleaner.
     But attackers can also directly fetch and read external .js files! --
>
```

3.2 DOM Manipulation — The Core of Interactive JavaScript

The DOM (Document Object Model) is the browser's internal tree representation of an HTML document. JavaScript manipulates the DOM to dynamically change what the user sees without reloading the page.

```
// Access an element by its id attribute
document.getElementById('demo')

// Change the text content of an element
document.getElementById('demo').innerHTML = 'Hack the Planet';
// Note: innerHTML renders HTML — use textContent to prevent HTML
injection!

// Change a CSS style property
document.getElementById('demo').style.color = 'red';

// Button with onclick that updates element text
<button onclick='document.getElementById("demo").innerHTML = "Button
Clicked";'>
  Click Me!
</button>

// JS function triggered by button click
function sayHi(name) {
  document.getElementById('demo').innerHTML = 'Hi ' + name;
  // DANGER: If 'name' comes from unsanitised user input, this is HTML
injection!
}
```

innerHTML vs textContent — A Critical Security Difference

innerHTML sets the content as HTML — any HTML tags in the string are rendered by the browser. If user input is passed directly to innerHTML without sanitisation, an attacker can inject their own HTML (HTML injection) or JavaScript (XSS). textContent treats the string as plain text — HTML tags are not interpreted. Always use textContent when displaying user-supplied data.

3.3 Lab — JavaScript DOM Manipulation

The room includes two hands-on JavaScript labs in the HTML+JS editor:

Lab	Task	Code Used	Flag
1	Add JavaScript that changes the demo element's content to 'Hack the Planet'.	`document.getElementById('demo').innerHTML = 'Hack the Planet';`	**JSISFUN**
2	Add a button that changes element text to 'Button Clicked' when clicked.	`<button onclick='getElementById("demo").innerHTML = "Button Clicked";'>Click Me!</button>`	(No flag — lab exercise)

4. Sensitive Data Exposure

Sensitive Data Exposure occurs when a website fails to properly protect or remove sensitive information from its client-side (front-end) code. Because all front-end code is sent to every user's browser and can be read by anyone, any credentials, API keys, tokens, or private links left in HTML, JavaScript, or code comments are fully exposed.

The Core Problem

Developers sometimes embed temporary credentials, debug information, or private URLs in client-side code during development — intending to remove them before deployment — and then forget. Since front-end source code is publicly readable by anyone who visits the page, this creates an immediate, critical vulnerability that requires no special tools to exploit.

4.1 What Sensitive Data Can Be Exposed

Type of Exposed Data	Impact & Example
Login credentials	Username and password left in HTML comments or JS variables. Direct authentication bypass — attacker logs in immediately.

API keys & tokens	Third-party service keys (AWS, Google Maps, Stripe) left in JS files. Allows attackers to abuse paid services or access data.
Hidden admin URLs	Links to admin panels or private pages commented out in HTML. Attacker navigates directly to the restricted area.
Database credentials	DB connection strings accidentally included in client-side code. Grants direct database access.
Internal infrastructure	Server names, internal IP addresses, version numbers, file paths. Provides roadmap for further attack.
Debug information	Stack traces, error messages, dev notes left in source or comments. Reveals code structure, libraries, and potential vulnerabilities.

4.2 How to Find Exposed Sensitive Data

- Right-click the page → 'View Page Source' (Ctrl+U) — see the raw HTML including all comments.
- Open Browser Developer Tools → Sources tab — browse all loaded JS, CSS, and HTML files.
- Use Ctrl+F in page source to search for keywords: password, passwd, key, token, secret, admin, login.
- Check JS files loaded by the page — they are publicly accessible and often contain API keys.
- Look at HTML comments (<!-- ... -->) — commonly where developers leave notes and forgotten credentials.

4.3 Lab — Finding the Hidden Password

The room provides a live website where sensitive data has been accidentally left in the page source code. The task is to find the hidden credentials using 'View Frame Source'.

```
<!-- Hidden in the HTML source code as a comment: -->
<!-- username: testpasswd -->

# Steps to find it:
# 1. Right-click on the page → 'View Frame Source'
# 2. Use Ctrl+F to search for 'password' or 'username'
# 3. Found: <!-- username: testpasswd -->
# 4. Use these credentials to log in to the site
```

Flag Captured — testpasswd

The password hidden in the page source code is testpasswd — found in an HTML comment. This demonstrates the most basic form of sensitive data exposure: a developer

left credentials in a comment that is visible to any visitor who checks the source. In a real-world assessment, this would be reported as a Critical finding.

4.4 Defences Against Sensitive Data Exposure

- Never embed credentials, API keys, tokens, or private URLs in any client-side code.
- Use environment variables or server-side configuration files for secrets — never hard-code them.
- Implement a pre-deployment review checklist that specifically checks for sensitive data in source code.
- Use automated scanning tools (git-secrets, TruffleHog, Semgrep) to scan code repositories for secrets.
- Remove all HTML comments before deploying to production — comments are visible to all users.
- Store API keys in a secrets manager (AWS Secrets Manager, HashiCorp Vault) — never in source code.
- Conduct regular security code reviews specifically looking for inadvertent data exposure.

5. HTML Injection

HTML Injection is a web vulnerability that occurs when a website takes user-supplied input and includes it directly in the page's HTML without sanitising it first. The browser renders the injected HTML as real page content — allowing an attacker to add their own elements, links, and text to the page as if they were part of the original website.

How HTML Injection Differs from XSS

HTML injection injects HTML elements (headings, links, images, forms). XSS (Cross-Site Scripting) injects JavaScript — which executes code in the victim's browser. HTML injection is the precursor and often escalates to XSS. Both exploit the same root cause: unsanitised user input rendered directly in the page.

5.1 How HTML Injection Works

The vulnerability arises when a JavaScript function takes user input and inserts it into the page using innerHTML (or equivalent) without stripping HTML tags:

```
<!-- Vulnerable HTML form -->
<div id='demo'></div>
<input type='text' id='name' placeholder="What's your name?">
<button onclick='sayHi()'>Say Hi!</button>
```

```
<!-- Vulnerable JavaScript function -->
<script>
  function sayHi() {
    var name = document.getElementById('name').value;
    // VULNERABLE: passing unsanitised user input directly to innerHTML
    document.getElementById('demo').innerHTML = 'Hi ' + name;
  }
</script>

<!-- Normal input: 'Alice' → Hi Alice (safe) -->
<!-- Malicious input: '<h1>Injected!</h1>' → renders as a real heading -->
<!-- Malicious link: '<a href="http://hacker.com">Click Here!</a>'
                  → renders as a real clickable link on the page -->
```

5.2 Lab — HTML Injection Attack

The room provides a vulnerable 'Say Hi' form. The task is to inject HTML so that a malicious link to http://hacker.com appears on the page:

Step	Detail
1. Open the form	Navigate to the vulnerable 'What's your name?' form on the lab page.
2. Enter the payload	Instead of a name, enter: Click Here!
3. Click Say Hi	The JavaScript function passes the input to innerHTML without sanitisation.
4. Browser renders	The browser renders the injected <a> tag as a real, clickable link on the page.
5. Flag revealed	The injected link appears and the flag HTML_INJ3CTI0N is revealed.

Flag Captured — HTML_INJ3CTI0N

Injecting Click Here! into the form renders a real hyperlink on the page, pointing to the attacker's site. This is the HTML_INJ3CTI0N flag. In a real attack, this link could be used for phishing — the injected link appears to be part of the trusted website, making users more likely to click it.

5.3 HTML Injection Attack Scenarios

Attack Type	What the Attacker Does & Why It Works
Phishing links	Injects Reset Password — the link appears to be part of the trusted site, increasing click-through rates for credential theft.
Defacement	Injects Hacked! or replaces content — visual defacement of the page for other users.
Fake login forms	Injects a fake <form> that POSTs credentials to the attacker's server. Users believe they are logging into the real site.
Image injection	Injects <img src='http://attacker.com/track.png'> — forces victims' browsers to make requests to the attacker's server, revealing IP addresses.
Escalation to XSS	If <script> tags are not filtered, HTML injection escalates to XSS — injected JavaScript executes in victims' browsers, enabling cookie theft and session hijacking.

5.4 Defences Against HTML Injection

- Never pass unsanitised user input to innerHTML. Use textContent or innerText for plain text output.
- Sanitise all user input server-side before storing or rendering it — strip or encode HTML tags.
- Use a trusted HTML sanitisation library (DOMPurify for JS) if you genuinely need to render user-supplied HTML.
- Implement Content Security Policy (CSP) headers to restrict what content the browser is allowed to render.
- Validate input on both client-side (for UX) and server-side (for security). Client-side validation is bypassable.
- Apply the principle of 'never trust user input' universally — every field, every parameter, every header.

6. All Lab Flags — Quick Reference

#	Task	What You Did	Flag / Answer
1	Front End vs Back End	Identified the correct term for the browser-rendered component.	`Front End`
2	HTML — Broken image	Fixed <img src='img/cat-2'> to <img src='img/cat-2.jpg'> in the editor.	`HTMLHERO`

3	HTML — Add dog image	Added <img src='img/dog-1.png'> on line 11.	(No flag)
4	JavaScript — DOM update	Used getElementById + innerHTML to change demo to 'Hack the Planet'.	JSISFUN
5	JavaScript — Button click	Added button with onclick that sets demo innerHTML to 'Button Clicked'.	(No flag)
6	Sensitive Data Exposure	Found <!-- username: testpasswd --> in page source via View Frame Source.	testpasswd
7	HTML Injection	Injected Click Here! into the sayHi form.	HTML_INJ3CTI0N

7. Security Implications

All Front-End Code Is Public — Always

There is no such thing as hidden client-side code. Every HTML file, JavaScript file, and CSS file your browser loads is fully readable. Any developer who assumes 'users won't look at the source' is making a fatal security error. Source code review is the very first thing an attacker or security assessor does.

innerHTML Is a Direct Path to Injection

Any code that passes user-controlled data to innerHTML without sanitisation is vulnerable to HTML injection — and potentially XSS. This is one of the most common web vulnerabilities found in production applications. Modern frameworks (React, Vue, Angular) escape HTML by default, which is one of the key security benefits of using them.

View Page Source Is a Free Recon Tool

Right-click → View Page Source reveals the entire HTML structure, all loaded JavaScript and CSS files, comments, hidden form fields, and metadata. From this, an attacker can identify: technologies used, external dependencies, potential API endpoints, admin paths, and any sensitive data exposure. This costs nothing and requires no special skills.

Developer Tools Go Deeper Than Source View

Browser Developer Tools (F12) go beyond page source — they show dynamically generated content, network requests, cookie values, localStorage, sessionStorage, WebSocket traffic, and

errors. Every web security professional and attacker uses browser developer tools as their primary inspection tool.

The 'Never Trust User Input' Rule

Every input field, URL parameter, HTTP header, and cookie value is potentially attacker-controlled. The HTML injection lab demonstrates what happens when this rule is violated — user input rendered as HTML without sanitisation. This principle extends to: SQL queries (SQL injection), shell commands (command injection), file paths (path traversal), and every other context where user data is used.

8. Key Takeaways & Glossary

8.1 Key Takeaways

- Every website has a Front End (rendered by browser — publicly visible) and a Back End (on the server — hidden).
- The three pillars of web development: HTML (structure), CSS (presentation), JavaScript (behaviour).
- HTML uses elements/tags to define page structure. All HTML is visible via 'View Page Source'.
- <!DOCTYPE html> declares HTML5. <head> holds metadata. <body> holds visible content.
- class attribute: multiple elements can share it. id attribute: unique per page, used by JS and CSS.
- JavaScript manipulates the DOM via document.getElementById() — changes pages without reloading.
- innerHTML renders HTML; textContent renders plain text. Use textContent for user-supplied data.
- Sensitive Data Exposure: credentials, tokens, API keys left in HTML comments or JS — all publicly readable.
- View Page Source shortcut: Ctrl+U (Windows/Linux), Cmd+Option+U (Mac), or right-click → View Page Source.
- HTML Injection: unsanitised user input passed to innerHTML allows attacker to inject HTML elements.
- Lab answers: HTMLHERO (broken image) → JSISFUN (JS task) → testpasswd (hidden cred) → HTML_INJ3CTI0N

8.2 Core Glossary

Term	Definition
Front End (Client-Side)	The part of a web application rendered by the user's browser — HTML, CSS, JavaScript. Publicly readable.

Back End (Server-Side)	The server-side code (Python, PHP, Node.js, databases) that processes requests. Never sent to the browser.
Web Server	A computer running software that receives HTTP requests and responds with web content.
HTML	HyperText Markup Language — the markup language that defines the structure and content of web pages.
CSS	Cascading Style Sheets — controls the visual presentation: colours, fonts, layout, spacing.
JavaScript	Programming language that adds interactivity and dynamic behaviour to web pages. Runs in the browser.
DOM	Document Object Model — the browser's live tree of HTML elements. JavaScript manipulates the DOM.
Element / Tag	The building blocks of HTML. e.g. <h1>, <p>, <img>, <a>. Define type and structure of content.
<!DOCTYPE html>	HTML5 document declaration — must be the first line of every HTML file.
id attribute	Unique identifier for one specific HTML element. Used by CSS and document.getElementById() in JS.
class attribute	Non-unique identifier shared by multiple elements. Used for CSS group styling.
innerHTML	JS property that gets/sets element content as HTML. DANGEROUS with user input — enables HTML injection.
textContent	JS property that gets/sets element content as plain text. SAFE for user-supplied data.
onclick	HTML event attribute — triggers JavaScript code when an element is clicked.
View Page Source	Browser feature (Ctrl+U) showing raw HTML sent by the server. First tool for web recon.
Sensitive Data Exposure	Vulnerability where secrets (credentials, tokens, API keys) are left in publicly readable front-end code.
HTML Injection	Vulnerability where unsanitised user input is rendered as HTML — allows injecting arbitrary page elements.
XSS	Cross-Site Scripting — HTML injection escalated to JavaScript injection; executes code in victims' browsers.
Sanitisation	The process of cleaning user input to remove or neutralise HTML tags, SQL syntax, or other dangerous characters.
Input Validation	Checking that user input matches expected format and constraints before processing it.

8.3 Next Steps on TryHackMe

- **Immediate:** Continue with the Putting It All Together room — combines DNS, HTTP, and web server concepts into a complete picture.
- **Security:** Complete the OWASP Top 10 room — HTML injection and sensitive data exposure are both OWASP categories.
- **Security:** Study XSS (Cross-Site Scripting) rooms — the dangerous escalation of HTML injection.
- **Essential:** Take Burp Suite Basics — intercept, inspect, and modify every HTTP request to a web application.
- **Path:** Explore the Web Fundamentals learning path for a complete grounding in web application security.

Every Web Vulnerability Starts Here

The concepts in this room — front end vs back end, HTML structure, JS DOM manipulation, sensitive data exposure, and HTML injection — are the foundations that every web security topic builds on. SQL injection, XSS, CSRF, IDOR, and authentication bypass are all variations of the same core principle: web applications process untrusted data, and when that processing is insecure, attackers win.

Source: TryHackMe — How Websites Work Room | tryhackme.com/room/howwebsiteswork